

SAP HANA SyBA Practice Build Runbook

Structured build guide, definitions, validation checks, likely exam variations, and virtual assessment environment

Prepared from the supplied SAP System Based Assessment build-style practice tasks

Important: This guide uses realistic practice scenarios only. It is not an official SAP exam dump, official SAP certification material, or a guarantee of actual exam content.

Document Breakdown

This is a solid SyBA-style practice set. To make it more executable and easier to validate, I would turn it into a build-and-check script/runbook like this.

- Section 1 - How SyBA-style grading works and what the examiner usually validates.
- Section 2 - Definitions of the key SAP HANA and assessment terms used in the tasks.
- Section 3 - The typical virtual environment used for the test, including tools and expected access paths.
- Section 4 - Execution strategy and naming rules to avoid automated validation failures.
- Section 5 - Build-by-build guide for the 15 supplied tasks.
- Section 6 - Exam-style variations that could appear with the same skill pattern.
- Section 7 - Final submission checklist.

Use the SQL examples as a practice template. Exact syntax can vary slightly by SAP HANA version, cockpit version, security policy, and whether the task is performed from SYSTEMDB or the tenant database.

1. How SyBA-Style Grading Works

In a System Based Assessment, the answer is not a written response. The answer is the completed object or configuration inside the live SAP system.

Automated grading commonly checks exact names, status, privileges, object definitions, inserted records, backup entries, and the dependency sequence. A small mismatch such as TrainingDB instead of TRAININGDB can cause a failed validation.

What is checked	Example
Exact object name	TRAININGDB, APP_USER, CUSTOMER_MASTER
Correct object type	Table versus view, restricted user versus normal user
Correct values	Exact column names, data types, data rows, backup prefix
Correct privileges	REPORT_USER can read the view but cannot modify data
Correct operational status	Tenant database final status is Running
Correct scope	Parameter changed at the required system, database, or tenant layer

Golden Rules

- Use the exact names from the task document.
- Understand whether you are connected to SYSTEMDB or the tenant database.
- Do not grant extra privileges unless the task requires them.
- Validate every build item with a query or cockpit check before moving on.
- Preserve the final required state. For example, after stop/start tasks, the tenant must end in Running status.

2. Definitions and Key Concepts

Term	Definition
SyBA	System Based Assessment. A practical assessment where you configure objects in a live or simulated SAP system. The system state is checked against expected results.
SYSTEMDB	The system database in an SAP HANA multiple-container database environment. Tenant administration such as creating, starting, stopping, and backing up tenants is usually performed here.
Tenant database	An isolated SAP HANA database container, such as TRAININGDB, where application schemas, users, tables, and views exist.
SYSTEM user	A highly privileged administrative user in SAP HANA. Each

	tenant database can have its own SYSTEM user.
SAP HANA cockpit	A browser-based administration and monitoring tool used to register resources, monitor services, manage users, configure parameters, and trigger backups.
Database Explorer / SQL Console	A tool used to connect to a HANA database and execute SQL statements.
Schema	A logical namespace for database objects. A database user typically owns a schema with the same name as the user, such as APP_USER.
Privilege	Permission to perform an action. Examples include SELECT on a view, CREATE ANY, or object privileges on tables and views.
Role	A collection of privileges that can be assigned to a user. Some SyBA tasks ask for direct privileges; others ask for roles.
Restricted user	A limited HANA user intended for narrow access. Restricted users typically require explicit object privileges and sometimes explicit client access privileges.
SQL view	A virtual table based on a SELECT statement. ACTIVE_CUSTOMERS_V filters CUSTOMER_MASTER to show only active customers.
Backup prefix	The name prefix used for a backup. Automated validation can search backup catalog entries for this exact prefix.
Audit policy	A configuration object that records selected auditable events, such as successful and unsuccessful login attempts.
Configuration layer	The scope at which an INI parameter is applied, such as SYSTEM, DATABASE, or tenant-specific scope.
HANA services	Runtime services such as nameserver and indexserver. Some services are optional or version-dependent.

3. Typical Virtual Environment for the Test

A SyBA test is normally completed in a controlled virtual lab. The exact vendor platform can differ, but the working pattern is usually similar.

Expected Access Pattern

1. Log in to the assessment portal using the provided credentials.
2. Open a remote desktop, virtual machine, cloud lab, or browser-based workbench.
3. Use SAP HANA cockpit for administration and monitoring tasks.
4. Use SAP HANA Database Explorer, SQL Console, HANA Studio, or an equivalent SQL client for SQL tasks.
5. Connect to SYSTEMDB for tenant lifecycle tasks.
6. Connect to TRAININGDB or the assigned tenant for user, schema, table, view, and audit tasks.
7. Complete all build tasks in the live system before selecting final submission.

Tools You May See

Tool	Likely purpose in the assessment
SAP HANA cockpit	Register cockpit resource, monitor services, manage users, run backup, check database status
Database Explorer / SQL Console	Run SQL to create users, tables, views, grants, audit policies, and validation queries
Remote desktop / browser VM	Controlled exam workspace where copy/paste, external internet, or local file access may be restricted
Terminal / command line	May be available for HDBSQL or OS-level checks, depending on the lab
Task document panel	Shows exact names, passwords, parameter values, and expected objects
Submission / validation button	Triggers automated checking of the live SAP system state

Environment Rules to Expect

- The database system may already exist, or the first task may ask you to create a tenant.
- Credentials are normally provided in the task document; do not invent names or passwords.
- The lab may reset if you log out or exceed the time limit.
- External documentation access may be blocked. Practice the sequence and validation queries beforehand.
- Some UI labels differ across cockpit versions, but the required end state is usually the same.
- Case sensitivity matters for quoted identifiers and exact validation names. Use the exact names shown in the task.

Connection Decision Table

Task type	Connect to
Create, stop, start, or backup tenant database	SYSTEMDB, unless the task explicitly says tenant context
Create APP_USER or REPORT_USER	TRAININGDB tenant
Create table/view and insert data as APP_USER	TRAININGDB tenant as APP_USER
Grant SELECT on APP_USER.ACTIVE_CUSTOMERS_V	TRAININGDB tenant as SYSTEM or privileged admin
Register cockpit resource	SAP HANA cockpit UI
Monitor HANA services	Cockpit monitoring view or SQL in relevant database context
Change database parameter	SYSTEMDB or tenant, depending on requested layer and scope

4. Execution Strategy Before You Build

Recommended Sequence

8. Create and verify the tenant database.
9. Connect to the tenant and create users.
10. Create APP_USER-owned objects and insert data.
11. Create the filtered view.
12. Create REPORT_USER and grant only the required read access.
13. Perform administration tasks such as backup, resource registration, monitoring, parameters, and stop/start.
14. Create audit policy and run final validation queries.
15. Re-check final state immediately before submission.

Password and Quoting Note

Password quoting rules can vary by SQL client and HANA syntax mode. The examples use a quoted password form because the supplied password contains a special character. If the lab SQL console rejects the quoted form, use the cockpit user creation form or the exact quoting style shown in the lab documentation.

Schema Qualification

When APP_USER creates CUSTOMER_MASTER, the object normally belongs to schema APP_USER. In validation SQL, prefer schema-qualified names:

```
APP_USER.CUSTOMER_MASTER
APP_USER.ACTIVE_CUSTOMERS_V
```

5. Build-by-Build Execution Guide

Each build below follows the same pattern: context, objective, steps, SQL or cockpit action, validation, expected result, common mistakes, and likely variations.

Build 1: Create a Tenant Database

Context: Connect to SYSTEMDB as a sufficiently privileged administrator.

Objective: Create tenant database TRAININGDB and leave it running.

Build Steps

16. Create the tenant database with the exact name TRAININGDB.
17. Set the SYSTEM user password to Welcome@12345.
18. Confirm that the tenant is active/running.

Command or Cockpit Action

```
CREATE DATABASE TRAININGDB SYSTEM USER PASSWORD "Welcome@12345";
```

Validation

```
SELECT DATABASE_NAME, ACTIVE_STATUS
FROM SYS.M_DATABASES
WHERE DATABASE_NAME = 'TRAININGDB';
```

Expected outcome: TRAININGDB exists and ACTIVE_STATUS indicates active/running.

Common Mistakes

- Creating TrainingDB instead of TRAININGDB.
- Running tenant creation in the tenant instead of SYSTEMDB.
- Forgetting to verify final running status.

Exam-Style Variations

- Different tenant name, for example SALESDB or ASSESSDB.
- Different SYSTEM password.
- Tenant must be created but initially stopped, then started later.

Build 2: Create a Database User

Context: Connect to TRAININGDB as SYSTEM or another user with user administration privileges.

Objective: Create APP_USER and ensure first-login password change is not required.

Build Steps

19. Create the user with the exact uppercase name APP_USER.
20. Use the required password.
21. Disable forced password change if the task asks for No.

Command or Cockpit Action

```
CREATE USER APP_USER PASSWORD "Welcome@12345" NO FORCE_FIRST_PASSWORD_CHANGE;
```

Validation

```
SELECT USER_NAME, USER_DEACTIVATED, IS_PASSWORD_CHANGE_NEEDED
FROM SYS.USERS
WHERE USER_NAME = 'APP_USER';
```

Expected outcome: APP_USER exists, is not deactivated, and does not require password change.

Common Mistakes

- Creating the user in SYSTEMDB instead of TRAININGDB.
- Forgetting NO FORCE_FIRST_PASSWORD_CHANGE.
- Typing app_user in lowercase if the validator expects uppercase.

Exam-Style Variations

- Create a user with a validity period.
- Create a user that must change password on first login.
- Create a technical user with a different name such as INTEGRATION_USER.

Build 3: Assign Roles or Privileges to APP_USER

Context: Connect to TRAININGDB as SYSTEM or privileged administrator.

Objective: Allow APP_USER to create and insert data in its own schema.

Build Steps

22. Use the minimum privileges required by the task.
23. If the task explicitly asks for CREATE ANY, grant it.
24. Validate by logging in as APP_USER and creating/inserting into a test object.

Command or Cockpit Action

-- Only use this if the task explicitly expects CREATE ANY:

```
GRANT CREATE ANY TO APP_USER;
```

-- Practical validation as APP_USER:

```
CREATE TABLE TEST_PRIV_CHECK (ID INTEGER);
```

```
INSERT INTO TEST_PRIV_CHECK VALUES (1);
```

```
DROP TABLE TEST_PRIV_CHECK;
```

Validation

```
SELECT *
FROM SYS.GRANTED_PRIVILEGES
WHERE GRANTEE = 'APP_USER';
```

Expected outcome: APP_USER can create objects and insert records in its own schema.

Common Mistakes

- Granting excessive privileges when the task expects least privilege.
- Not testing as APP_USER.
- Granting privileges in the wrong tenant.

Exam-Style Variations

- Create a custom role and assign it to APP_USER.
- Grant only object-level privileges on a provided schema.
- Revoke extra privileges before final validation.

Build 4: Create CUSTOMER_MASTER Table

Context: Connect to TRAININGDB as APP_USER.

Objective: Create CUSTOMER_MASTER with exactly four columns and required data types.

Build Steps

25. Create the table in APP_USER schema.
26. Use exact column names and data types.
27. Do not add extra columns unless the task asks.

Command or Cockpit Action

```
CREATE TABLE CUSTOMER_MASTER (  
    CUSTOMER_ID INTEGER,  
    CUSTOMER_NAME NVARCHAR(100),  
    CITY NVARCHAR(50),  
    STATUS NVARCHAR(20)  
);
```

Validation

```
SELECT COLUMN_NAME, DATA_TYPE_NAME, LENGTH  
FROM SYS.TABLE_COLUMNS  
WHERE SCHEMA_NAME = 'APP_USER'  
    AND TABLE_NAME = 'CUSTOMER_MASTER'  
ORDER BY POSITION;
```

Expected outcome: CUSTOMER_MASTER exists with CUSTOMER_ID, CUSTOMER_NAME, CITY, and STATUS.

Common Mistakes

- Using VARCHAR instead of NVARCHAR.
- Adding a primary key if not requested.
- Creating the table as SYSTEM rather than APP_USER.

Exam-Style Variations

- Add a primary key on CUSTOMER_ID.
- Add an EMAIL or COUNTRY column.
- Create the table in a named schema instead of the user schema.

Build 5: Insert Records into CUSTOMER_MASTER

Context: Connect to TRAININGDB as APP_USER.

Objective: Insert exactly the three required customer records.

Build Steps

28. Insert values exactly as provided.
29. Check count and row values.
30. Do not insert duplicate or test records unless you remove them.

Command or Cockpit Action

```
INSERT INTO CUSTOMER_MASTER  
    (CUSTOMER_ID, CUSTOMER_NAME, CITY, STATUS)  
VALUES  
    (1001, 'Alpha Ltd', 'London', 'Active');
```

```

INSERT INTO CUSTOMER_MASTER
  (CUSTOMER_ID, CUSTOMER_NAME, CITY, STATUS)
VALUES
  (1002, 'Beta Corp', 'Berlin', 'Active');

INSERT INTO CUSTOMER_MASTER
  (CUSTOMER_ID, CUSTOMER_NAME, CITY, STATUS)
VALUES
  (1003, 'Delta Inc', 'Paris', 'Inactive');

```

Validation

```

SELECT *
FROM CUSTOMER_MASTER
ORDER BY CUSTOMER_ID;

SELECT COUNT(*) AS RECORD_COUNT
FROM CUSTOMER_MASTER;

```

Expected outcome: Exactly three rows exist with the specified values.

Common Mistakes

- Using ACTIVE instead of Active.
- Leaving extra test rows in the table.
- Misspelling customer names or cities.

Exam-Style Variations

- Different rows and statuses.
- Insert only active customers, then update one to inactive.
- Load rows from a CSV or import tool instead of SQL insert statements.

Build 6: Create ACTIVE_CUSTOMERS_V SQL View

Context: Connect to TRAININGDB as APP_USER.

Objective: Create a view that returns only customers with STATUS = Active.

Build Steps

31. Create the view with the exact required name.
32. Use a WHERE clause for STATUS = 'Active'.
33. Verify that only the active customer rows are returned.

Command or Cockpit Action

```

CREATE VIEW ACTIVE_CUSTOMERS_V AS
SELECT
  CUSTOMER_ID,
  CUSTOMER_NAME,
  CITY,
  STATUS

```



```
FROM CUSTOMER_MASTER
WHERE STATUS = 'Active';
```

Validation

```
SELECT *
FROM ACTIVE_CUSTOMERS_V
ORDER BY CUSTOMER_ID;
```

Expected outcome: The view returns CUSTOMER_ID 1001 and 1002 only.

Common Mistakes

- Filtering with lowercase active when values use Active.
- Creating the view in the wrong schema.
- Using SELECT * is usually acceptable but explicit columns are safer for validation.

Exam-Style Variations

- Filter by CITY, for example CITY = 'London'.
- Create inactive customers view.
- Create a view with renamed columns or only selected columns.

Build 7: Create Restricted REPORT_USER

Context: Connect to TRAININGDB as SYSTEM or privileged administrator.

Objective: Create REPORT_USER with read-only access only to ACTIVE_CUSTOMERS_V.

Build Steps

34. Create REPORT_USER as a restricted user.
35. Grant SELECT on APP_USER.ACTIVE_CUSTOMERS_V only.
36. Grant restricted client access if required by the lab connection method.
37. Validate that REPORT_USER cannot create or modify objects.

Command or Cockpit Action

```
CREATE RESTRICTED USER REPORT_USER
PASSWORD "Welcome@12345"
NO FORCE_FIRST_PASSWORD_CHANGE;

GRANT SELECT ON APP_USER.ACTIVE_CUSTOMERS_V TO REPORT_USER;

-- If required by your client:
GRANT RESTRICTED_USER_JDBC_ACCESS TO REPORT_USER;

-- or:
GRANT RESTRICTED_USER_ODBC_ACCESS TO REPORT_USER;
```

Validation

```
SELECT *
FROM SYS.GRANTED_PRIVILEGES
WHERE GRANTEE = 'REPORT_USER';

-- Test as REPORT_USER:
```

```
SELECT * FROM APP_USER.ACTIVE_CUSTOMERS_V;
```

Expected outcome: REPORT_USER can select from the view but cannot create tables, insert, update, or delete.

Common Mistakes

- Granting SELECT on CUSTOMER_MASTER when only the view is required.
- Creating a normal user instead of a restricted user.
- Forgetting client access privilege for restricted user connectivity.

Exam-Style Variations

- REPORT_USER can read a table instead of a view.
- REPORT_USER can only read one city or one status via a view.
- Create a role called REPORT_ROLE and assign the role.

Build 8: Configure a Complete Data Backup

Context: Usually connect to SYSTEMDB for tenant backup, or use cockpit backup tile.

Objective: Perform a complete data backup of TRAININGDB using prefix TRAININGDB_FULL_BACKUP.

Build Steps

38. Open the backup function in cockpit or use SQL if enabled.
39. Choose complete data backup.
40. Use the exact backup prefix.
41. Wait for completion before moving to validation.

Command or Cockpit Action

```
-- From SYSTEMDB:
BACKUP DATA FOR TRAININGDB USING FILE ('TRAININGDB_FULL_BACKUP');

-- If connected directly to TRAININGDB and permitted:
BACKUP DATA USING FILE ('TRAININGDB_FULL_BACKUP');
```

Validation

```
SELECT ENTRY_TYPE_NAME, STATE_NAME, SYS_START_TIME, SYS_END_TIME, DESTINATION_PATH
FROM SYS.M_BACKUP_CATALOG
WHERE DESTINATION_PATH LIKE '%TRAININGDB_FULL_BACKUP%'
ORDER BY SYS_START_TIME DESC;
```

Expected outcome: A successful complete data backup exists with prefix TRAININGDB_FULL_BACKUP.

Common Mistakes

- Using a different prefix.
- Submitting before backup completes.
- Backing up SYSTEMDB instead of TRAININGDB.

Exam-Style Variations

- Different backup prefix.
- Log backup instead of full data backup.
- Backup through cockpit only, not SQL.

Build 9: Check Backup Status

Context: Use cockpit backup catalog or SQL backup catalog view.

Objective: Verify the requested backup completed successfully.

Build Steps

42. Search the backup catalog for the exact prefix.
43. Confirm the latest matching backup state is successful/completed.
44. Confirm there is no failed entry for the required prefix after your run.

Command or Cockpit Action

```
SELECT ENTRY_ID, ENTRY_TYPE_NAME, STATE_NAME, SYS_START_TIME, SYS_END_TIME
FROM SYS.M_BACKUP_CATALOG
WHERE DESTINATION_PATH LIKE '%TRAININGDB_FULL_BACKUP%'
ORDER BY SYS_START_TIME DESC;
```

Validation

Same as the backup status query.

Expected outcome: Latest matching backup entry shows successful or completed.

Common Mistakes

- Checking only the UI message and not the catalog.
- Filtering on the wrong prefix.
- Confusing old successful backup with your required backup.

Exam-Style Variations

- Validate from cockpit only.
- Find failed backup entries and identify cause.
- Check backup duration or destination path.

Build 10: Create a Cockpit Resource

Context: Use SAP HANA cockpit UI.

Objective: Register the database system as HANA_TRAINING_RESOURCE.

Build Steps

45. Open cockpit database directory or resource registration area.
46. Register the target HANA system using the lab host and credentials.
47. Set resource name exactly to HANA_TRAINING_RESOURCE.
48. Confirm resource status is available.

Command or Cockpit Action

-- This is normally a cockpit UI task, not a SQL task.

Validation

Cockpit -> Database Directory / Resources -> confirm HANA_TRAINING_RESOURCE exists and is available.

Expected outcome: A managed cockpit resource named HANA_TRAINING_RESOURCE is available for monitoring and administration.

Common Mistakes

- Registering with a similar but not exact resource name.
- Creating a resource for the wrong tenant or host.
- Stopping after registration without checking availability.

Exam-Style Variations

- Register multiple resources.
- Update an existing resource name.
- Assign the resource to a group or monitoring collection.

Build 11: Monitor Database Services

Context: Use cockpit monitoring view or SQL.

Objective: Verify required HANA services are visible and running where applicable.

Build Steps

49. Open service monitoring in cockpit.
50. Check nameserver and indexserver at minimum.
51. Check xsengine only if XS classic is used.
52. Document any optional services that are not present in the system version.

Command or Cockpit Action

```
SELECT DATABASE_NAME, HOST, SERVICE_NAME, ACTIVE_STATUS, SQL_PORT
FROM SYS.M_SERVICES
ORDER BY DATABASE_NAME, SERVICE_NAME;
```

Validation

Confirm nameserver, indexserver, compileservers, preprocessor, and xsengine if used.

Expected outcome: Required services are running where applicable.

Common Mistakes

- Marking xsengine missing as an error when XS classic is not enabled.
- Checking the wrong database context.
- Ignoring failed or inactive services.

Exam-Style Variations

- Identify a stopped service.
- Check service memory or CPU in cockpit.
- Filter services by tenant database.

Build 12: Change a Database Parameter

Context: Use cockpit configuration UI or SQL with correct scope.

Objective: Change the requested parameter at the correct layer and scope.

Build Steps

53. Read the task document carefully for file, section, parameter, value, and layer.
54. Apply the change at SYSTEM, DATABASE, or tenant level as required.
55. Use WITH RECONFIGURE if the parameter supports online reconfiguration.
56. Validate exact value from configuration views.

Command or Cockpit Action

```
ALTER SYSTEM ALTER CONFIGURATION ('global.ini', 'SYSTEM')
SET ('memorymanager', 'global_allocation_limit') = '<VALUE_FROM_TASK>'
WITH RECONFIGURE;

-- Tenant-specific examples vary by HANA version and cockpit workflow.
-- Always match the task layer and scope.
```

Validation

```
SELECT FILE_NAME, LAYER_NAME, DATABASE_NAME, SECTION, KEY, VALUE
FROM SYS.M_INIFILE_CONTENTS
WHERE FILE_NAME = 'global.ini'
AND SECTION = 'memorymanager'
AND KEY = 'global_allocation_limit';
```

Expected outcome: The requested parameter value appears at the correct layer and scope.

Common Mistakes

- Changing SYSTEM when the task asks for tenant scope.
- Entering wrong units or value format.
- Changing a similar parameter in the wrong section.

Exam-Style Variations

- Change a trace parameter instead of memorymanager.
- Change parameter through cockpit only.
- Reset a parameter back to default after validation.

Build 13: Stop and Start a Tenant Database

Context: Connect to SYSTEMDB or use cockpit administration.

Objective: Stop TRAININGDB, verify stopped, start it again, and leave it running.

Build Steps

57. Stop the tenant database.
58. Verify the tenant is stopped/inactive.
59. Start the tenant database again.
60. Verify the final status is running.

Command or Cockpit Action

```
ALTER SYSTEM STOP DATABASE TRAININGDB;

SELECT DATABASE_NAME, ACTIVE_STATUS
FROM SYS.M_DATABASES
WHERE DATABASE_NAME = 'TRAININGDB';

ALTER SYSTEM START DATABASE TRAININGDB;

SELECT DATABASE_NAME, ACTIVE_STATUS
```

```
FROM SYS.M_DATABASES
WHERE DATABASE_NAME = 'TRAININGDB';
```

Validation

Final query shows TRAININGDB active/running.

Expected outcome: TRAININGDB is stopped and restarted successfully; final status is Running.

Common Mistakes

- Forgetting to start the tenant again.
- Stopping the wrong database.
- Trying to run tenant tasks while it is stopped.

Exam-Style Variations

- Restart a different tenant.
- Stop and start from cockpit only.
- Start an already stopped tenant and validate status.

Build 14: Create an Audit Policy

Context: Connect to TRAININGDB or required administration context with audit privileges.

Objective: Create AUDIT_APP_USER_LOGIN to audit successful and unsuccessful login attempts for APP_USER.

Build Steps

61. Confirm auditing is enabled if the task requires active audit logging.
62. Create an audit policy with the exact policy name.
63. Target successful and unsuccessful connect/login events for APP_USER.
64. Validate policy existence.

Command or Cockpit Action

```
CREATE AUDIT POLICY AUDIT_APP_USER_LOGIN
AUDITING SUCCESSFUL CONNECT, UNSUCCESSFUL CONNECT
USER APP_USER
LEVEL INFO;

-- If auditing must be enabled:
ALTER SYSTEM ALTER CONFIGURATION ('global.ini', 'SYSTEM')
SET ('auditing configuration', 'global_auditing_state') = 'true'
WITH RECONFIGURE;
```

Validation

```
SELECT *
FROM SYS.AUDIT_POLICIES
WHERE POLICY_NAME = 'AUDIT_APP_USER_LOGIN';
```

Expected outcome: AUDIT_APP_USER_LOGIN exists and tracks login activity for APP_USER.

Common Mistakes

- Creating policy for REPORT_USER instead of APP_USER.
- Auditing only successful logins when both successful and unsuccessful are required.

- Wrong policy name.

Exam-Style Variations

- Audit SELECT on a table or view.
- Audit failed logins for all users.
- Create policy then test login and check audit log entries.

Build 15: Validate Final Configuration

Context: Use SYSTEMDB, TRAININGDB, cockpit, and SQL checks as appropriate.

Objective: Confirm all required objects exist with exact names, correct configuration, and correct operational status.

Build Steps

65. Run database status checks.
66. Run user, table, view, and data checks.
67. Run privilege checks.
68. Run backup catalog checks.
69. Confirm cockpit resource and service monitoring status.
70. Remove unintended test objects or extra records.

Command or Cockpit Action

```
-- SYSTEMDB:
SELECT DATABASE_NAME, ACTIVE_STATUS
FROM SYS.M_DATABASES
WHERE DATABASE_NAME = 'TRAININGDB';

-- TRAININGDB:
SELECT USER_NAME
FROM SYS.USERS
WHERE USER_NAME IN ('APP_USER', 'REPORT_USER')
ORDER BY USER_NAME;

SELECT SCHEMA_NAME, TABLE_NAME
FROM SYS.TABLES
WHERE SCHEMA_NAME = 'APP_USER'
AND TABLE_NAME = 'CUSTOMER_MASTER';

SELECT SCHEMA_NAME, VIEW_NAME
FROM SYS.VIEWS
WHERE SCHEMA_NAME = 'APP_USER'
AND VIEW_NAME = 'ACTIVE_CUSTOMERS_V';

SELECT * FROM APP_USER.CUSTOMER_MASTER ORDER BY CUSTOMER_ID;
SELECT * FROM APP_USER.ACTIVE_CUSTOMERS_V ORDER BY CUSTOMER_ID;
```

Validation

All required objects exist with exact names and expected state.

Expected outcome: TRAININGDB, APP_USER, REPORT_USER, CUSTOMER_MASTER, ACTIVE_CUSTOMERS_V, backup prefix, and cockpit resource are all correct.

Common Mistakes

- Leaving extra rows or objects that change validation results.
- Not checking REPORT_USER negative permissions.
- Forgetting cockpit resource validation because it is UI-based.

Exam-Style Variations

- Create an evidence checklist before submission.
- Use a single validation script.
- Identify and fix one intentionally misconfigured object.

6. Additional Exam-Style Variations to Practice

The following variations keep the same skill pattern but change the names, values, sequence, or validation target. Practicing these makes the tasks less dependent on memorization.

Skill area	Possible variation	What to watch
Tenant lifecycle	Create tenant SALESDB, stop it, start it, then leave it running.	Final status matters more than intermediate status.
User management	Create TECH_USER with password change required on first login.	Do not use NO FORCE_FIRST_PASSWORD_CHANGE if task says Yes.
Privileges	Grant SELECT only on one view through a custom role.	Avoid direct table access if only view access is requested.
Table creation	Create PRODUCT_MASTER with DECIMAL, DATE, and NVARCHAR columns.	Exact data type, precision, scale, and column order may be checked.
Data load	Insert five records and update one status.	Final data state must match expected values exactly.
View logic	Create a view for inactive customers or customers in one city.	String matching and case can affect results.
Restricted user	REPORT_USER can use JDBC but not ODBC.	Grant only the required restricted access privilege.
Backup	Run a backup with prefix ASSESS_FULL_BACKUP.	Prefix, database, and backup type are all checked.
Monitoring	Find which service is stopped or has high memory use.	Some services are optional by HANA version.
Parameter	Change a parameter at tenant layer and not system layer.	Layer/scope is a common grading point.
Audit	Audit unsuccessful logins for all users.	Policy target and event type must match exactly.
Cleanup	Drop an incorrectly named object and recreate with exact name.	Validators may fail on duplicate or wrong objects if final state is ambiguous.

7. Final Validation Script Template

Use this as a final pre-submission checklist. Run the relevant sections in the correct database context.

```
-- Run in SYSTEMDB:
SELECT DATABASE_NAME, ACTIVE_STATUS
FROM SYS.M_DATABASES
```



```

WHERE DATABASE_NAME = 'TRAININGDB';

SELECT ENTRY_TYPE_NAME, STATE_NAME, SYS_START_TIME, SYS_END_TIME, DESTINATION_PATH
FROM SYS.M_BACKUP_CATALOG
WHERE DESTINATION_PATH LIKE '%TRAININGDB_FULL_BACKUP%'
ORDER BY SYS_START_TIME DESC;

-- Run in TRAININGDB:
SELECT USER_NAME, USER_DEACTIVATED, IS_PASSWORD_CHANGE_NEEDED
FROM SYS.USERS
WHERE USER_NAME IN ('APP_USER', 'REPORT_USER')
ORDER BY USER_NAME;

SELECT COLUMN_NAME, DATA_TYPE_NAME, LENGTH
FROM SYS.TABLE_COLUMNS
WHERE SCHEMA_NAME = 'APP_USER'
  AND TABLE_NAME = 'CUSTOMER_MASTER'
ORDER BY POSITION;

SELECT * FROM APP_USER.CUSTOMER_MASTER ORDER BY CUSTOMER_ID;
SELECT * FROM APP_USER.ACTIVE_CUSTOMERS_V ORDER BY CUSTOMER_ID;

SELECT *
FROM SYS.GRANTED_PRIVILEGES
WHERE GRANTEE IN ('APP_USER', 'REPORT_USER')
ORDER BY GRANTEE;

SELECT *
FROM SYS.AUDIT_POLICIES
WHERE POLICY_NAME = 'AUDIT_APP_USER_LOGIN';

SELECT DATABASE_NAME, HOST, SERVICE_NAME, ACTIVE_STATUS
FROM SYS.M_SERVICES
ORDER BY DATABASE_NAME, SERVICE_NAME;

```

8. Final Submission Checklist

- TRAININGDB exists and final status is Running.
- APP_USER exists in TRAININGDB and can log in without first password change.
- REPORT_USER exists as restricted user where required.
- APP_USER.CUSTOMER_MASTER exists with exact columns and data types.
- CUSTOMER_MASTER contains exactly the required records and no extras.
- APP_USER.ACTIVE_CUSTOMERS_V returns only active customers.
- REPORT_USER can SELECT from APP_USER.ACTIVE_CUSTOMERS_V.
- REPORT_USER cannot create tables or modify data.
- TRAININGDB_FULL_BACKUP exists in the backup catalog and completed successfully.

- HANA_TRAINING_RESOURCE exists in SAP HANA cockpit and is available.
- Required HANA services are running where applicable.
- Any required parameter change is at the correct layer and scope.
- AUDIT_APP_USER_LOGIN exists and targets APP_USER login activity.
- No temporary validation objects remain unless explicitly allowed.

Study Approach

Practice each build in isolation first, then run the full sequence end-to-end. During the test, work from exact names and expected outcomes, not from memory alone. The safest pattern is build, validate, correct, then proceed.

For this specific scenario, the highest-risk grading points are tenant context, exact object names, REPORT_USER least privilege, backup prefix, parameter layer/scope, and final TRAININGDB running status.